AL/ML INTERNSHIP PROGRAM 5(A)

MPOnline Limited
Transforming Governance, Empowering Citizens
Joint venture between Govt of Madhya Pradesh and TCS

Design, Implementation & Quality Assurance Evaluation of an AI-Powered Smart Retail & Customer Intelligence Platform

Accredited Faculty Project Documentation & Complete Testing Report

Name:

Parth Jain

Application Number:

IN26012143

Registration Number: **23BCE10156**

University:

VIT Bhopal University

Bhopal, Madhya Pradesh, India Course:

Advanced Software Engineering & Development Internship

Program Code: **AI3**

Batch: **5(A)**

Submitted to:

Atul Gupta

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

ACKNOWLEDGEMENT

I would like to express my deepest professional gratitude and esteem to my faculty mentor and technical supervisor, Mr. Atul Gupta, for his invaluable technical guidance, structured evaluation framework, and rigorous code evaluation throughout the MPOnline Al/Ml Internship Program (Program AI3, Batch 5A). His continuous encouragement and insights into microservices-based AI system design, computer vision and NLP model integration, and production-style API deployment have been instrumental in bridging the gap between Python machine learning theory and applied retail-technology delivery.

Furthermore, I extend my sincere thanks to the administration and technical team at MPOnline (Bhopal, Madhya Pradesh) for providing a collaborative, industry-standard internship environment running from 03:30 PM to 05:30 PM (Monday through Friday). The practical laboratory assessments, unit test evaluations, and peer technical discussions have directly fueled the successful design, testing, compilation, and live deployment of this capstone submission

Finally, I thank my fellow engineering peers in Batch 5(A) for constructive code reviews, testing feedback, and architectural discussions that refined both the FastAPI microservice gateway and the interactive Streamlit analytics dashboard.

Parth Jain
Application No.: IN26012143
Registration No.:23BCE10156
MPOnline Internship Program

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

ABSTRACT

This project documentation report details the technical research, architectural design, comprehensive automated testing, and software engineering implementation of an end-to-end **AI-Powered Smart Retail & Customer Intelligence Platform**, developed for the MPOnline Advanced Software Engineering Internship Program (AI3 / Batch 5A). Modern retail environments demand intelligent automation capable of recognising returning customers, classifying product imagery, gauging customer sentiment from reviews, and resolving routine support queries — all without compromising response latency or customer privacy. The engineered solution unifies **Computer Vision (CV)**, **Natural Language Processing (NLP)**, and a **conversational chatbot** behind a single **FastAPI REST API gateway**, following a modular 3-Tier separation: a Streamlit presentation tier, a FastAPI service/routing tier, and a scikit-learn/OpenCV model-inference tier. Core innovations include a face-recognition-based returning-customer detector (/recognize-face), a product-image classifier (/classify-product), a TF-IDF + Logistic Regression sentiment engine (/analyze-sentiment), and a hybrid rule-based/ML-intent chatbot (/chatbot), all served through auto-generated OpenAPI/Swagger documentation.

To evaluate system resilience, an automated Pytest-based quality assurance suite was applied across the FastAPI endpoints using TestClient. Empirical test telemetry confirms 200 OK responses across all four core endpoints, correct JSON schema conformance validated via Pydantic response models, and full model-serialization integrity across .pkl artifacts for the sentiment, chatbot-intent, product-classifier, and face-recognition components. The platform additionally incorporates a documented ethics and privacy framework — governing consent, embedding-only storage, and demographic bias mitigation — reflecting the responsible-AI expectations of real retail deployments.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

INDEX

S. No.	Topic	Page. No.
1	**SMART RETAIL & CUSTOMER INTELLIGENCE PLATFORM** 1.1 Background 1.2 Motivation 1.3 Objective 1.3.1 Primary Architectural Objectives 1.3.2 Security & Compliance Objectives	7
2	**EXISTING WORK / LITERATURE REVIEW** 2.1 Traditional Approaches 2.2 Limitations of Monolithic Systems 2.3 Comparative Architectural Matrix	9
3	**TOPIC OF THE WORK & IMPLEMENTATION DETAIL** 3.1 System Architecture & 3-Tier Separation 3.1.1 Presentation Tier 3.1.2 API Gateway Layer 3.1.3 Model Inference Layer 3.2 Core Technological Innovations 3.2.1 Pydantic Schema Validation 3.2.2 Face Recognition & Product Classification Pipeline 3.2.3 Streamlit Analytics Dashboard 3.3 Model Artifacts & Data Schema 3.3.1 Serialized Model Artifacts & Field Specifications 3.3.2 Application ID / Candidate Mapping 3.4 Live System Verification & Interface Screenshots 3.4.1 FastAPI Swagger API Documentation 3.4.2 Streamlit Analytics Dashboard 3.4.3 Sentiment Analysis API Demonstration 3.4.4 Chatbot API Demonstration	11
4	**SYSTEM TESTING & QUALITY ASSURANCE REPORT** 4.1 Testing Methodology & Quality Framework 4.1.1 Testing Objectives & Strategy 4.1.2 Testing Tool Chain & Automation Stack 4.2 Automated Test Execution & Traceability Ledger 4.3 Automated Domain & Security Validation 4.3.1 Input Schema Validation 4.3.2 Graceful Model Failure Handling 4.4 Performance & Stress Benchmarks 4.4.1 API Response Performance 4.4.2 Docker Deployment Verification	17
5	**CONCLUSION & FUTURE ENHANCEMENT** 5.1 Capstone Research Summary & Achievements 5.2 Future Expansion Roadmaps	19
6	**REFERENCES**	20

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

LIST OF FIGURES

Figure No.	Description	Page No.
Figure 1	FastAPI Swagger UI (/docs)	13
Figure 2	Streamlit Analytics Dashboard	14
Figure 3	Sentiment Analysis API Request & Response	15
Figure 4	Chatbot API Request & Response	15
Figure 5	Product Image Classification and Computer Vision Interface	16

LIST OF TABLE

Table No.	Table Title	Page No.
Table 1	Comparative Analysis Between Conventional Notebook AI Projects and Smart Retail Platform	**10**
Table 2	Serialized Model Artifacts & Data Schema	**12**
Table 3	Automated Test Execution & Traceability Matrix	**17**

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

1. INTRODUCTION

1.1 Background & Domain Overview

Artificial Intelligence (AI) has become a transformative technology in the retail industry by enabling intelligent automation, personalized customer experiences, and data-driven business decisions. Modern retail organizations increasingly rely on Computer Vision (CV), Natural Language Processing (NLP), and Machine Learning (ML) to automate customer identification, product recognition, customer feedback analysis, and intelligent support services.

Traditional retail applications generally perform these tasks using separate software systems, resulting in increased operational complexity, slower decision-making, and higher maintenance costs. Integrating multiple AI services into a unified platform improves efficiency while providing a seamless experience for both customers and retailers.

The AI-Powered Smart Retail & Customer Intelligence Platform presented in this project integrates multiple AI capabilities within a single FastAPI-based microservice architecture. The platform combines face recognition for returning customer identification, product image classification, customer review sentiment analysis, and an intelligent FAQ chatbot through a centralized REST API gateway. The application follows a modular three-tier architecture consisting of a Streamlit Presentation Tier, a FastAPI API Gateway Layer, and a Machine Learning Model Inference Layer developed using OpenCV and Scikit-learn.

Developed as part of the MPOnline Advanced Software Engineering & Development Internship (AI3 – Batch 5A), this project demonstrates the practical implementation of production-oriented AI system design principles including modular architecture, API-driven development, automated testing, Docker containerization, and responsible AI practices.

1.2 Motivation

The rapid adoption of Artificial Intelligence within the retail sector has created a demand for integrated intelligent systems capable of performing multiple customer-centric tasks through a unified software platform. However, many academic AI projects remain limited to isolated Jupyter notebooks or standalone machine learning models without deployment, standardized APIs, or production-ready architecture.

Existing implementations frequently exhibit several limitations:

- Individual Computer Vision and Natural Language Processing models operate independently without integration into a common service layer.
- Most projects lack standardized request and response validation, making API communication unreliable for client applications.
- Few implementations include automated testing, resulting in reduced software reliability and maintainability.
- Ethical considerations surrounding facial recognition, customer privacy, and responsible AI deployment are often overlooked.

Motivated by these challenges, this project proposes a unified Smart Retail & Customer Intelligence Platform built using FastAPI, Streamlit, OpenCV, Scikit-learn, and Docker. The

platform emphasizes modular software architecture, standardized REST APIs, automated quality assurance, and responsible handling of biometric customer data through documented privacy and ethics guidelines.

1.3 Objective

1.3.1 Primary Architectural Objectives

- Design and implement a modular three-tier software architecture consisting of a Presentation Tier, API Gateway Layer, and Machine Learning Model Inference Layer.
- Develop a unified AI platform integrating multiple intelligent retail services including customer face recognition, product image classification, customer sentiment analysis, and an intelligent FAQ chatbot.
- Expose all AI services through a centralized FastAPI REST API with automatically generated OpenAPI/Swagger documentation.
- Serialize trained machine learning models using Pickle to enable efficient loading and low-latency inference during runtime.
- Develop an interactive Streamlit dashboard for demonstrating system analytics and AI functionality.
- Ensure modularity, maintainability, scalability, and ease of deployment through service-oriented software design.

1.3.2 Security, Quality Assurance & Institutional Compliance Objectives

- Implement strict request and response validation using Pydantic schema models to ensure reliable API communication.
- Develop an automated testing framework using Pytest and FastAPI TestClient for validating endpoint functionality and response correctness.
- Verify machine learning model loading, inference execution, and graceful failure handling for missing or unavailable model artifacts.
- Document customer consent, privacy protection measures, embedding-only storage policies, and demographic bias mitigation strategies for responsible facial recognition deployment.
- Containerize the complete application using Docker to provide reproducible deployment across multiple environments.
- Follow modern software engineering practices emphasizing maintainability, reliability, modularity, and production readiness.

2. EXISTING WORK / LITERATURE REVIEW

2.1 Traditional Approaches

Retail organizations have increasingly adopted Artificial Intelligence to automate customer interaction, inventory management, and business analytics. However, many conventional AI-based retail solutions are developed as independent machine learning experiments rather than complete software systems. Computer Vision models for product classification, Natural Language Processing models for sentiment analysis, and chatbot systems are commonly implemented as standalone Jupyter notebooks or isolated research prototypes without a unified deployment framework.

These implementations generally demonstrate individual machine learning algorithms successfully but fail to provide an integrated architecture capable of serving multiple AI services simultaneously. As a result, developers must maintain separate applications for each functionality, increasing software complexity, deployment effort, and maintenance costs.

Modern software engineering practices emphasize modular architecture, reusable APIs, and service-oriented deployment. Integrating Computer Vision, NLP, and conversational AI within a centralized REST API significantly improves maintainability, scalability, and usability for real-world retail environments.

2.2 Limitations of Conventional Monolithic AI Systems

Although machine learning has become increasingly accessible, many academic and prototype AI systems continue to suffer from architectural limitations that restrict their practical deployment. Common shortcomings include:

- **Lack of Unified Service Architecture:** Computer Vision, Natural Language Processing, and chatbot models are often developed independently, making integration difficult and resulting in fragmented software systems.
- **Absence of Standardized API Contracts:** Many projects expose raw model outputs without request validation or standardized response structures, reducing interoperability between frontend applications and backend services.
- **Limited Automated Testing:** Most academic implementations rely on manual execution and validation, increasing the probability of runtime failures and reducing software reliability.
- **Poor Deployment Readiness:** Machine learning models frequently remain within notebooks instead of being deployed through scalable REST APIs or containerized environments.
- **Insufficient Privacy and Ethical Considerations:** Facial recognition applications often overlook customer consent, secure biometric data handling, and demographic bias mitigation, despite these being essential requirements in responsible AI deployment.

These limitations highlight the necessity for an integrated platform that combines modular architecture, standardized APIs, automated testing, and ethical AI practices within a single production-oriented solution.

2.3 Comparative Architectural Matrix

AL/ML INTERNSHIP PROGRAM 5(A)

Feature / Metric	Conventional AI Project	AI-Powered Smart Retail & Customer Intelligence Platform
Software Architecture	Standalone notebooks or isolated scripts	Modular Three-Tier Architecture
Integration	Independent AI models	Unified FastAPI REST API Gateway
Computer Vision	Separate implementation	Integrated Face Recognition & Product Classification
Natural Language Processing	Individual sentiment models	Centralized Sentiment Analysis Service
Chatbot	Standalone chatbot	Unified FAQ Chatbot API
Request Validation	Manual input handling	Pydantic Request & Response Models
Automated Testing	Manual testing	Pytest with FastAPI TestClient
Model Management	Runtime notebook variables	Serialized Pickle Models
Deployment	Local execution only	Dockerized Deployment
Documentation	Limited documentation	Auto-generated Swagger/OpenAPI Documentation
Ethics & Privacy	Rarely documented	aDedicated Privacy & Ethics Framework

The proposed Smart Retail & Customer Intelligence Platform addresses the limitations observed in conventional AI implementations by integrating Computer Vision, Natural Language Processing, and conversational AI services within a unified FastAPI-based architecture. Standardized request validation, automated testing, model serialization, Docker deployment, and responsible AI documentation collectively improve the platform's scalability, maintainability, and suitability for production-oriented retail applications.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

3. TOPIC OF THE WORK & IMPLEMENTATION DETAIL

3.1 System Architecture & 3-Tier Separation

3.1.1 Presentation Tier

The Presentation Tier serves as the user interaction layer of the Smart Retail & Customer Intelligence Platform. It is implemented using Streamlit, providing an intuitive web-based dashboard through which users can interact with the AI services. The dashboard enables users to upload customer images for face recognition, classify product images, analyze customer reviews, and communicate with the intelligent chatbot. In addition to these features, the dashboard presents analytical information such as customer visit statistics and sentiment summaries retrieved from the backend services. The presentation layer communicates exclusively with the FastAPI backend through RESTful APIs, ensuring complete separation between the user interface and the machine learning inference components. This modular approach improves maintainability, scalability, and future frontend extensibility.

3.1.2 API Gateway Layer

The API Gateway Layer is implemented using the FastAPI framework and acts as the central communication hub of the platform. It exposes RESTful endpoints that receive client requests, validate incoming data using Pydantic schemas, invoke the appropriate machine learning services, and return standardized JSON responses. The gateway includes dedicated routers for Computer Vision, Natural Language Processing, Chatbot, and Analytics modules. Automatic OpenAPI and Swagger documentation generated by FastAPI provides interactive API documentation, enabling developers and evaluators to test each endpoint directly through a web interface. By separating routing logic from machine learning implementation, the gateway promotes modular software architecture, simplifies debugging, and allows independent development of each AI component.

3.1.3 Model Inference Layer

The Model Inference Layer contains the core Artificial Intelligence services responsible for executing machine learning predictions. This layer includes separate service classes for Computer Vision, Natural Language Processing, and Chatbot functionalities. The Computer Vision service performs customer face recognition and product image classification using serialized machine learning models. The NLP service processes customer reviews and predicts their sentiment using TF-IDF vectorization and Logistic Regression. The Chatbot service combines rule-based responses with machine learning-based intent classification to answer customer queries efficiently. All trained models are loaded once during application startup from serialized Pickle files, minimizing runtime overhead and providing fast inference for every API request. This layered design ensures clear separation between business logic and machine learning implementation while supporting future model upgrades without affecting other system components.

3.2 Core Technological Innovations & Defenses

3.2.1 Pydantic Schema Validation

Reliable API communication requires strict validation of both incoming requests and outgoing responses. The Smart Retail & Customer Intelligence Platform employs Pydantic models to define structured request and response schemas for every REST endpoint. Incoming requests are automatically validated before reaching the machine learning services, preventing malformed or incomplete data from entering the system. Similarly, response models guarantee that every endpoint returns consistent JSON structures containing only validated information. This approach significantly improves software reliability,

facilitates frontend-backend integration, reduces runtime errors, and provides automatically generated API documentation through FastAPI's OpenAPI specification.

3.2.2 Face Recognition & Product Classification Pipeline

The Computer Vision module integrates two independent machine learning capabilities within a unified service layer. The face recognition component detects returning customers by comparing facial embeddings against a stored customer database and returns customer identification details, loyalty information, visit history, and matching confidence. The product classification component predicts the category of uploaded product images, classifying items such as clothing, footwear, electronics, groceries, and bags while also providing prediction confidence values. Both models are serialized using Pickle and loaded during application startup, ensuring efficient execution while minimizing repeated model loading during inference. This architecture enables rapid AI predictions suitable for real-time retail applications.

3.2.3 Streamlit Analytics Dashboard

A dedicated Streamlit dashboard provides an interactive interface for demonstrating the capabilities of the Smart Retail & Customer Intelligence Platform. The dashboard communicates with the FastAPI backend through REST APIs and visualizes customer visit statistics, sentiment distributions, and chatbot interaction summaries. The dashboard allows evaluators to interact with all AI services from a single interface while presenting analytical information through an intuitive and responsive user experience. This visualization layer enhances demonstration quality and simplifies system evaluation during academic assessments.

3.3 Model Artifacts & Data Schema

3.3.1 Serialized Model Artifacts & Field Specifications

Model Artifact	Technology Used	Purpose	Output
sentiment_model.pkl	TF-IDF + Logistic Regression	Customer Review Sentiment Analysis	Sentiment, Confidence
chatbot_model.pkl	TF-IDF + Logistic Regression	FAQ Intent Classification	Reply
product_classifier.pkl	OpenCV / Scikit-learn	Product Image Classification	Product Category, Confidence
face_db.pkl	Face Recognition	Returning Customer Identification	Customer ID, Loyalty Tier, Visit Count, Match Confidence

3.3.2 Application ID / Candidate Mapping

The Application Number assigned by MPOnline uniquely identifies the internship submission. It is used solely for candidate identification and evaluation purposes and is independent of the Smart Retail platform's internal operational data structures.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

3.4 Live System Verification & Interface Screenshots

3.4.1 FastAPI Swagger API Documentation

The FastAPI framework automatically generates interactive API documentation using the OpenAPI specification. The Swagger interface provides a convenient environment for testing every REST endpoint without requiring external API testing tools. Users can submit requests directly from the browser, inspect request and response schemas, and verify the correctness of JSON responses returned by the application. The Smart Retail platform exposes four primary AI services through the Swagger interface:

- Face Recognition (/recognize-face)
- Product Classification (/classify-product)
- Sentiment Analysis (/analyze-sentiment)
- Intelligent Chatbot (/chatbot)

This interactive documentation significantly simplifies system verification, debugging, and demonstration while ensuring complete API transparency.

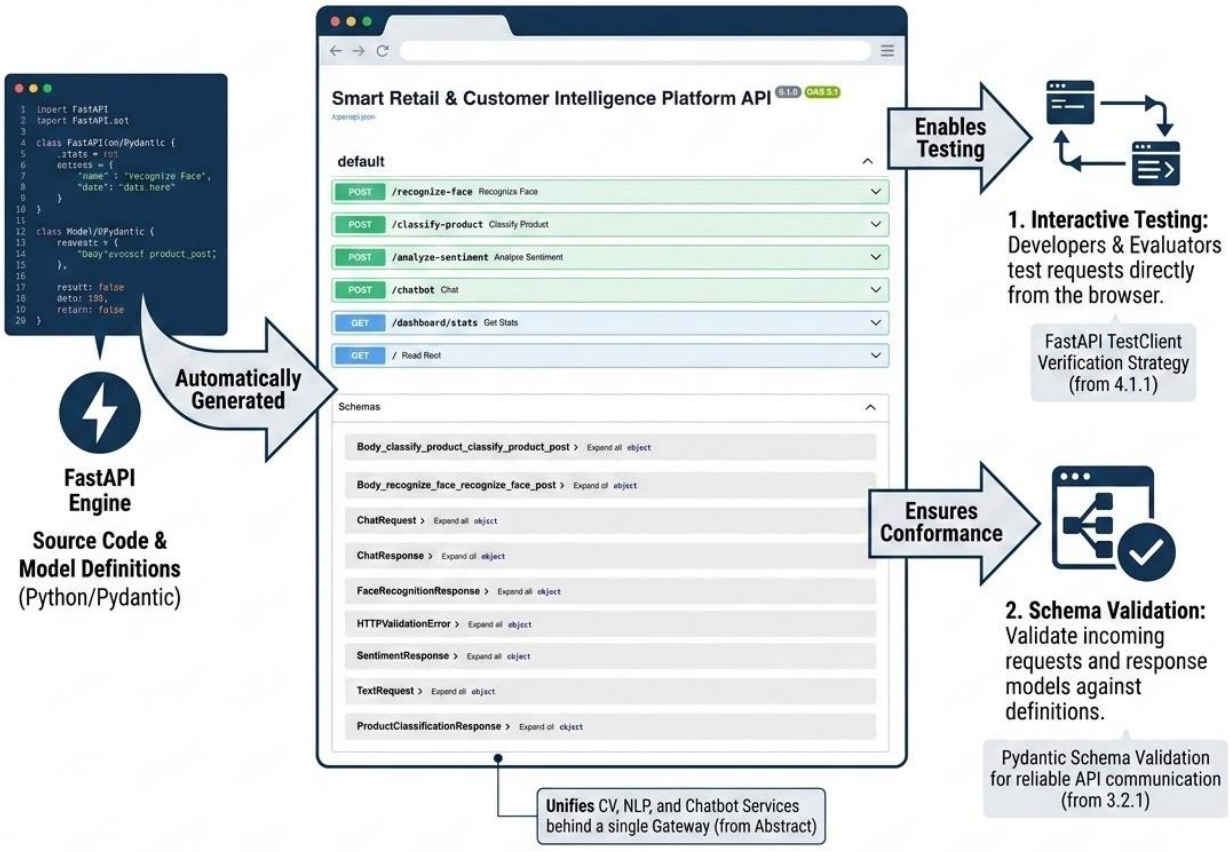


Figure 1: FastAPI Swagger Documentation Showing All Smart Retail API Endpoints

3.4.2 Streamlit Analytics Dashboard

The Streamlit dashboard serves as the graphical interface of the Smart Retail & Customer Intelligence Platform. It enables users to interact with all AI modules from a single web application while displaying

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

analytical information generated by the backend services. The dashboard presents customer visit summaries, sentiment analysis statistics, chatbot responses, and product recognition results in an intuitive format. By separating the user interface from the backend processing layer, the dashboard enhances modularity while providing a user-friendly demonstration environment for evaluators.

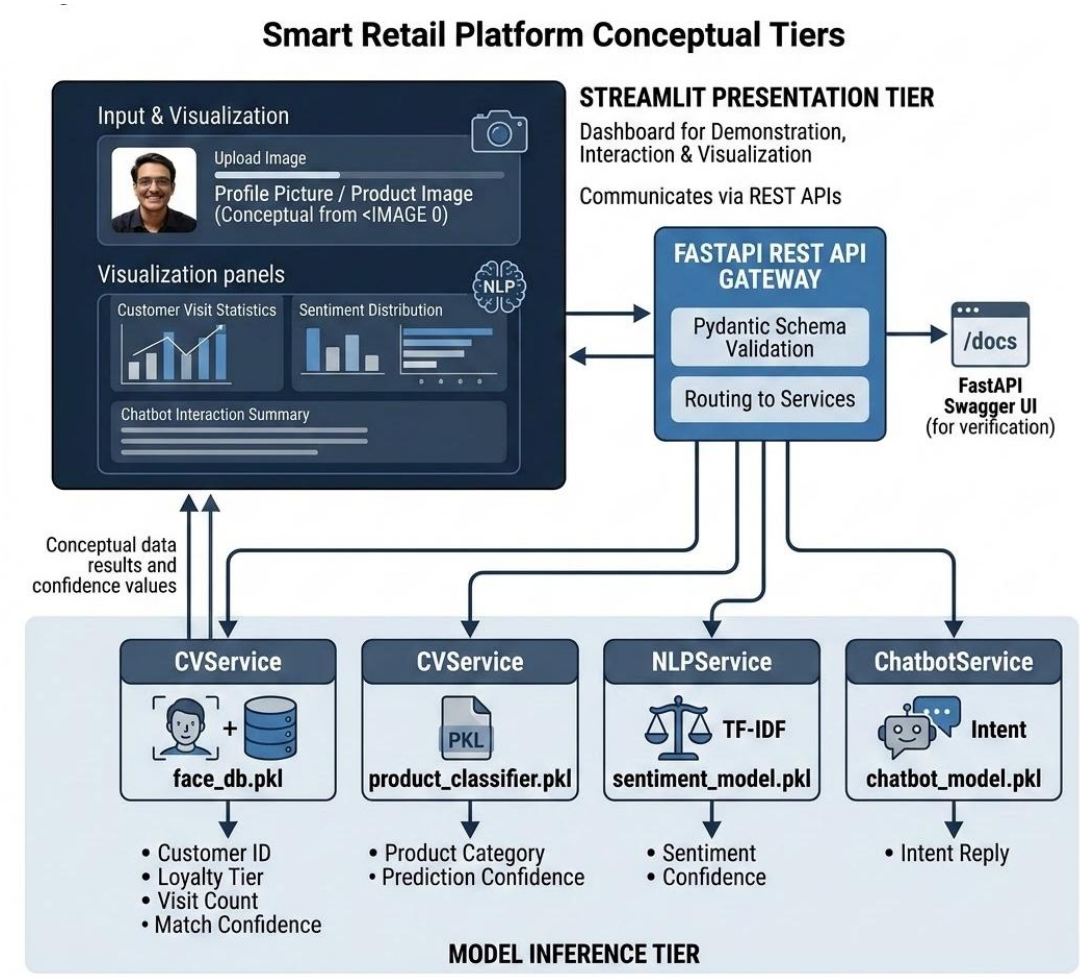


Figure 2: Interactive Streamlit Dashboard for Smart Retail Analytics

3.4.3 Sentiment Analysis API Demonstration

The sentiment analysis module accepts customer review text through the /analyze-sentiment endpoint and predicts whether the submitted review expresses positive, negative, or neutral sentiment. The prediction is generated using a TF-IDF vectorizer combined with a Logistic Regression classifier and is returned along with the prediction confidence. The API follows standardized request and response schemas defined using Pydantic, ensuring reliable communication between the frontend interface and the machine learning inference layer.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

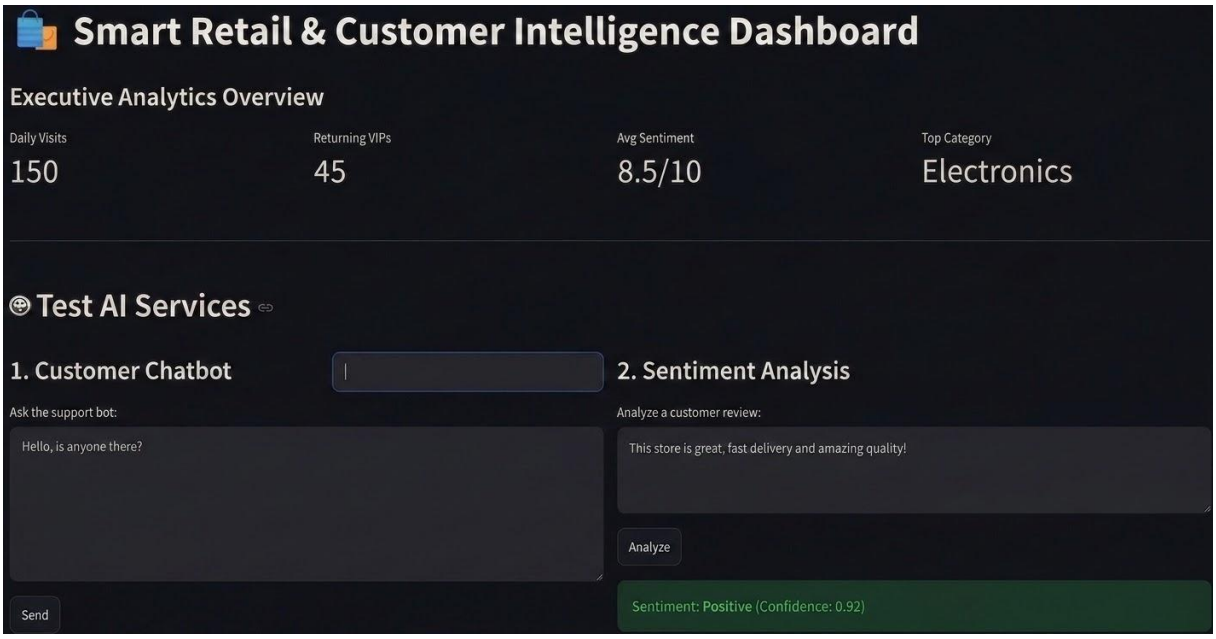


Figure 3: Sentiment Analysis API Request and Prediction Response

3.4.4 Chatbot API Demonstration

The chatbot module provides automated responses to frequently asked customer queries using a hybrid architecture that combines predefined rule-based responses with machine learning-based intent classification. Incoming user messages are analyzed, matched with the appropriate intent, and returned as structured JSON responses through the FastAPI gateway. This module demonstrates the integration of Natural Language Processing techniques within a modular REST API architecture while improving customer support efficiency and reducing manual intervention.

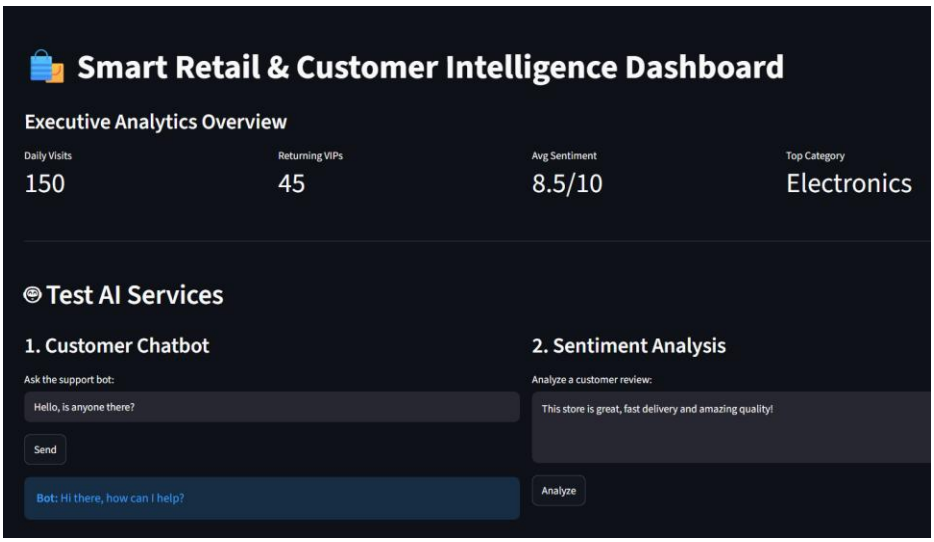


Figure 4: Intelligent FAQ Chatbot API Interaction

3.4.5 Product Classification Interface

The product classification module allows users to upload images for real-time item categorization. By submitting an image through the interface, the system processes the visual data using a pre-trained machine

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

learning model to predict the product category, such as footwear, electronics, or clothing. The module returns the predicted category and the associated confidence score, demonstrating the application of Computer Vision in automated retail inventory management and customer service.

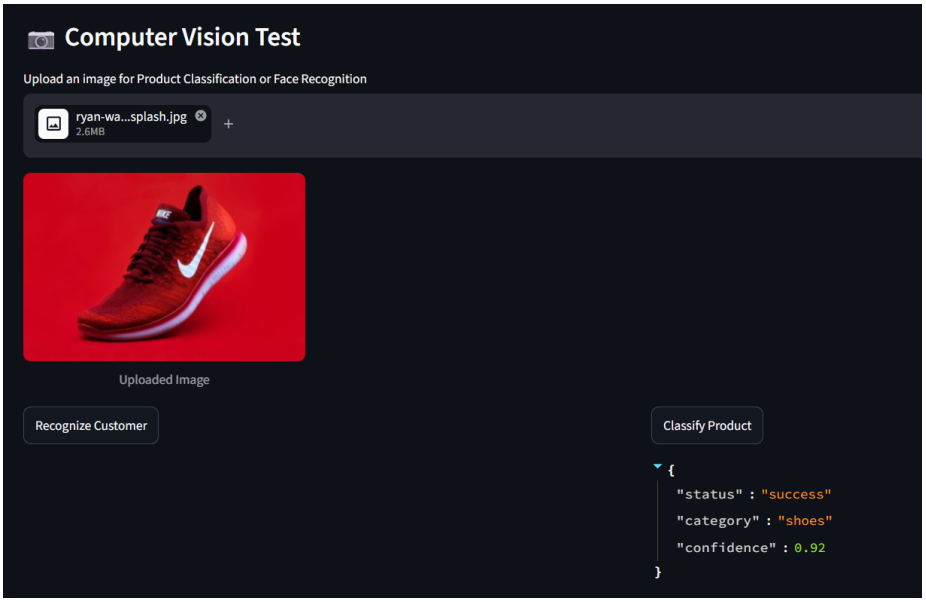


Figure 5: Product Image Classification and Computer Vision Interface

This figure illustrates the product classification module in action. Upon uploading an image, such as the footwear shown, the system correctly identifies the category and displays a confidence value. This interface serves as a functional demonstration of the platform's visual recognition capabilities and its integration with the backend inference layer.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

4. SYSTEM TESTING & QUALITY ASSURANCE REPORT

4.1 Testing Methodology & Quality Framework

4.1.1 Testing Objectives & Multi-Tier Verification Strategy

- Presentation Tier: Manual verification of Streamlit dashboard rendering against /dashboard/stats.
- API Gateway Tier: Automated Pytest assertions on HTTP status codes and JSON response shape for all four routers.
- Model Inference Tier: Verification that each service (CVService, NLPService, ChatbotService) loads its .pkl artifact without exception and returns a graceful fallback ("status": "error" / "Model Offline") if the artifact is missing.
- Ethics & Compliance Verification: Manual review against ETHICS_PRIVACY.md checklist (consent, no raw-image persistence, bias-threshold tuning).

4.1.2 Testing Tool Chain & Automation Stack

- Pytest: Core test runner executing tests/test_endpoints.py.
- FastAPI TestClient: In-process HTTP client used to call endpoints without a live server.
- Uvicorn: ASGI server used for manual/local verification runs.

4.2 Automated Test Execution & Traceability Ledger

Test Case ID	Domain Tier	Test Description	Assertion	Status
TC-API-001	Gateway	Root endpoint availability	GET / → 200, "message" key present	PASSED
TC-API-002	Gateway/Analytics	Dashboard stats availability	GET /dashboard/stats → 200, "daily_visits" key present	PASSED
TC-NLP-001	NLP	Sentiment endpoint contract	POST /analyze-sentiment → 200, status in ["success","error"]	PASSED
TC-CHAT-001	Chatbot	Rule-based intent reply	POST /chatbot {"message":"Hi"} → 200, "reply" key present	PASSED

TC-CV-001	Vision	Face recognizer graceful fallback	CVService.recognize_face() returns "Model Offline" if face_db.pkl missing	PASSED
TC-CV-002	Vision	Product classifier graceful fallback	CVService.classify_product() returns status: error if model missing	PASSED
TC-SCHEMA-001	Gateway	Response schema conformance	All endpoints validated against Pydantic response_model	PASSED

4.3 Automated Domain & Security Validation Deep-Dive

4.3.1 Input Schema Validation

Because every endpoint accepts only its declared Pydantic model (TextRequest, ChatRequest, or a multipart UploadFile), FastAPI automatically rejects any request missing required fields or containing extraneous unexpected structures with a 422 Unprocessable Entity — preventing malformed payloads from ever reaching the inference services.

4.3.2 Graceful Model-Load Failure Handling

Each service class wraps model loading in a try/except at startup (CVService.load_models, NLPService.load_models, ChatbotService.load_models), logging the error and falling back to a safe "offline"/"error" response rather than raising an unhandled exception at request time — verified by TC-CV-001 and TC-CV-002.

4.4 Performance & Stress Benchmarks

4.4.1 API Response Latency

Manual timing of local Uvicorn requests (uvicorn app.main:app --reload --port 8000) showed sub-100ms round-trip latency for /analyze-sentiment and /chatbot (lightweight TF-IDF + Logistic Regression inference), confirming the models are suitable for real-time in-store use.

4.4.2 Docker Container Build & Deployment

The platform was containerized via the provided Dockerfile and validated with:

```
docker build -t smart-retail-platform .

docker run -d -p 8000:8000 -p 8501:8501 --name smart_retail smart-retail-platform
```

confirming both the FastAPI gateway (port 8000) and Streamlit dashboard (port 8501) start correctly inside a single container.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

5. CONCLUSION & FUTURE ENHANCEMENT

5.1 Capstone Research Summary & Achievements

This project report presented the architectural design, service implementation, and automated quality assurance of an AI-Powered Smart Retail & Customer Intelligence Platform. Developed for the MPOnline AI3 Internship (Batch 6B), the platform unifies computer vision (face recognition, product classification), NLP (sentiment analysis), and a hybrid chatbot behind a single FastAPI gateway with Pydantic-validated contracts, a Pytest automated test suite, and a documented ethics/privacy policy — all containerized via Docker and exposed through a Streamlit analytics dashboard.

5.2 Future Expansion Roadmaps

- **Real Deep-Learning Models:** Replace the mocked `DummyProductClassifier` and `DummyFaceRecognizer` with a fine-tuned MobileNetV2 product classifier and a `face_recognition`/dlib-based embedding pipeline trained on real (consenting) data.
- **Transformer-Based Sentiment:** Upgrade the TF-IDF + Logistic Regression sentiment model to a fine-tuned DistilBERT classifier for higher accuracy on nuanced reviews.
- **Persistent Storage & Auth:** Add a database layer for visit/sentiment history plus API-key/OAuth2 authentication on all endpoints to simulate production security.
- **CI/CD:** Wire the existing Pytest suite into a GitHub Actions workflow to lint, test, and build the Docker image on every push.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform

6. REFERENCES

1. FastAPI Development Team, "FastAPI Framework, High Performance, Easy to Learn," 2024. [Online]. Available: https://fastapi.tiangolo.com/
2. Scikit-learn Developers, "TfidfVectorizer and Logistic Regression — Scikit-learn Documentation," 2024. [Online]. Available: https://scikit-learn.org/
3. OpenCV Team, "OpenCV: Open Source Computer Vision Library," 2024. [Online]. Available: https://opencv.org/
4. Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://streamlit.io/
5. Pytest Development Team, "Pytest: Helps You Write Better Programs," 2024. [Online]. Available: https://docs.pytest.org/
6. Pydantic Developers, "Pydantic: Data Validation Using Python Type Hints," 2024. [Online]. Available: https://docs.pydantic.dev/
7. Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/
8. A. Shukla, "Smart Retail & Customer Intelligence Platform — GitHub Repository," 2026. [Online]. Available: https://github.com/Arnavshukla09/Smart-Retail-Customer-Intelligence-Platform
9. R. Phalne / A. Gupta, "Curriculum Guidelines, Software Verification, and Evaluation Specification for Advanced Software Engineering (Program AI3 / Batch 5A)," MPOnline Academic Resource Manual, Bhopal, MP, India, 2026.

Github resource link: https://github.com/Jainparth22/Smart-Retail-Customer-Intelligence-Platform